

ARDUINO IOT PROJECT
SMART DOOR ALARMING SYSTEM
(DISTANCE PRESENCE)

9 JANUARY 2025

MUHAMAD HAIKAL

Table of Contents

Introduction 3
a) Literature Review 3
b) Project Objectives 3
Methodology 4
a) Source Code 4
b) IoT Project Methodology 6
i) Hardware 6
ii) Software 8
ii) Circuit Setup 8
Project Flowchart 9
Conclusion & Discussion 10
References 11

1.0 INTRODUCTION

a) Literature Review

With the rise in demand for enhanced home security systems, the integration of IoT technology has gained significant traction. Traditional alarm systems often operate in isolation, offering limited functionality. However, the emergence of IoT solutions has revolutionized this domain by enabling real-time monitoring, control, and scalability. The use of sensors, such as ultrasonic and environmental sensors, has proven to be highly effective in creating versatile and reliable systems. Recent studies have highlighted how combining multiple types of sensors can improve the accuracy and responsiveness of security systems, making them more efficient in detecting unauthorized access or unusual activity.

b) Project Objectives

The project main goal is to design and implement a "**Smart Door Alarming System"** that utilizes IoT technologies to improve home security. The specific project objectives are :

- To accurately detect the presence of a person near a door using an ultrasonic sensor.
- To trigger an audible alarm using a buzzer and show message/notification through an LED display upon detecting a person.
- To display real-time temperature and humidity data of the surrounding door area for environmental monitoring and helps user to assess conditions that may affect comfort, safety and functionality of the system.

2.0 METHODOLOGY

Below is the complete source code for the project with detailed comments for better understanding of the circuit :

a) Source Code

```
#include <Wire.h> // Include library for I2C communication
#include <LiquidCrystal_I2C.h> // Include library for controlling I2C LCD displays
#include <DHT.h> // Include library for DHT temperature and humidity sensor

// Define pin assignments for components
#define TRIG_PIN 9 // Pin for triggering the ultrasonic sensor
#define ECHO_PIN 10 // Pin for receiving the echo signal from the ultrasonic sensor
#define BUZZER_PIN 8 // Pin for controlling the buzzer

#define LED_PIN 5 // Pin for controlling the LED
#define DHT_PIN 7 // Pin connected to the DHT sensor

// Initialize the DHT sensor (DHT11 model)
DHT dht(DHT_PIN, DHT11);

// Initialize the LCD object with I2C address and display size
LiquidCrystal_I2C lcd(0x27, 16, 2);

// Variables for ultrasonic sensor readings
long duration; // Variable to store the time it takes for the echo to return
int distance;  // Variable to store the calculated distance in centimeters

void setup() {
  pinMode(TRIG_PIN, OUTPUT); // Set the ultrasonic trigger pin as output
  pinMode(ECHO_PIN, INPUT);  // Set the ultrasonic echo pin as input
  pinMode(BUZZER_PIN, OUTPUT); // Set the buzzer pin as output
  pinMode(LED_PIN, OUTPUT); // Set the LED pin as output

  // Initialize the DHT sensor
  dht.begin();

  // Initialize the LCD display
  lcd.begin(16, 2);    // Specify the LCD dimensions (16 columns x 2 rows)
  lcd.backlight();     // Turn on the LCD backlight for visibility
}
```

```
void loop() {
  // **Configuration 1: Measure distance using the ultrasonic sensor**
  digitalWrite(TRIG_PIN, LOW);    // Ensure the trigger pin starts LOW
  delayMicroseconds(2);           // Wait for 2 microseconds
  digitalWrite(TRIG_PIN, HIGH);   // Send a HIGH pulse to trigger the ultrasonic signal
  delayMicroseconds(10);          // Keep the trigger HIGH for 10 microseconds
  digitalWrite(TRIG_PIN, LOW);    // Set the trigger pin back to LOW

  // Measure the time of the echo signal
  duration = pulseIn(ECHO_PIN, HIGH); // Capture the duration of the HIGH echo pulse
  distance = duration * 0.034 / 2;    // Convert duration to distance in centimeters

  // **Configuration 2: Read temperature and humidity from the DHT11 sensor**
  float temp = dht.readTemperature(); // Get temperature in Celsius
  float humidity = dht.readHumidity(); // Get humidity as a percentage

  // **Configuration 3: Display temperature and humidity on the LCD**
  lcd.clear(); // Clear the previous content on the LCD
  lcd.setCursor(0, 0); // Move the cursor to the first row
  lcd.print("Temp: " + String(temp) + "C"); // Display the temperature
  lcd.setCursor(0, 1); // Move the cursor to the second row
  lcd.print("Humidity: " + String(humidity) + "%"); // Display the humidity

  // **Configuration 4: Check if an object is detected within range**
  if (distance < 100) { // If distance is less than 100 cm
    digitalWrite(LED_PIN, HIGH); // Turn on the LED
    digitalWrite(BUZZER_PIN, HIGH); // Activate the buzzer

    // Display detection message on the LCD
    lcd.setCursor(0, 0); // Move cursor to the first row
    lcd.print("Person Detected"); // Display detection message
    lcd.setCursor(0, 1); // Move cursor to the second row
    lcd.print("Infront The Door"); // Display location message

    delay(2000); // Keep the buzzer and LED on for 2 seconds

    // Turn off the LED and buzzer after 2 seconds
    digitalWrite(LED_PIN, LOW);
    digitalWrite(BUZZER_PIN, LOW);
  }

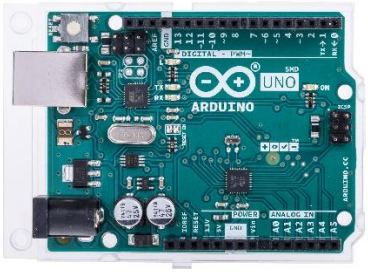
  delay(500); // Short delay before the next reading
}
```

b) <u>IoT Project Methodology</u>

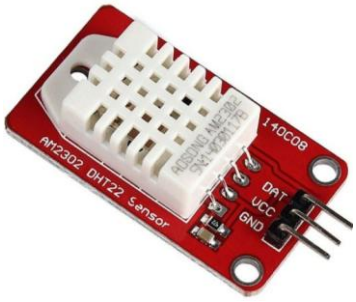
i) Hardware

This project utilizes a selection of hardware components that are suitable for Arduino setup.

1. Arduino UNO : The Arduino UNO is the central microcontroller that controls all components in the system. It processes inputs from the sensors, triggers outputs like the buzzer and LED, and manages data displayed on the LCD. It runs the code and communicates with other components through its digital and analog pins.



2. DHT11 Sensor : The DHT22 is a temperature and humidity sensor. It measures the ambient air conditions, providing higher precision in temperature and humidity readings. The Arduino reads the data from this sensor and displays it on the LCD.



3. I2C LCD Display : The I2C LCD display is used to show real-time information from the system. It outputs the temperature and humidity data received from the DHT22 sensor and also displays messages like when the ultrasonic sensor detects an object in front of the door.



4. HC-SR04 Ultrasonic Sensor : Measures the distance between the sensor and any object in its path using sound waves. The sensor emits a pulse (TRIG) and listens for the echo (ECHO) to determine the distance. If a person is detected within a defined range it triggers the alarm system.

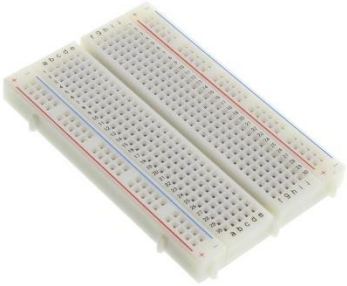


5. Buzzer and Resistor : The buzzer emits a sound when the ultrasonic sensor detects a person. This acts as an alert. A resistor is used with the buzzer to limit the current flowing through it, preventing damage to the components.





6. Breadboard and Jumper Wires : The breadboard is used to organize and connect all the components as described above. Jumper wires are used to make connections between the Arduino and other components. They are flexible and easy to use, allowing for quick and easy circuit assembly.





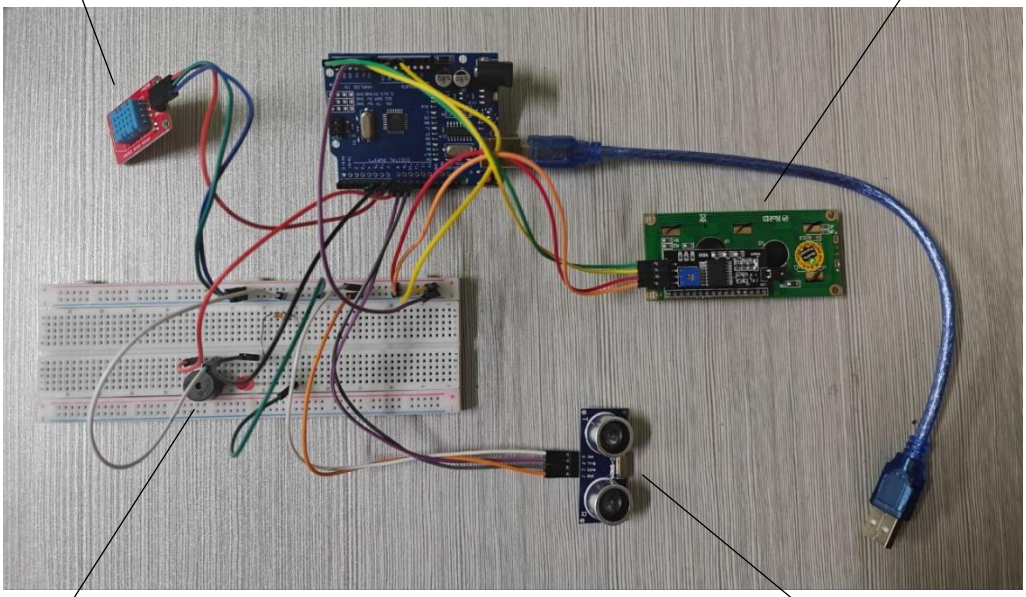
ii) Software

1. Arduino IDE : The Arduino Integrated Development Environment (IDE) is used for writing, compiling, and uploading the program (code) to the Arduino UNO. It provides a simple interface to communicate with the Arduino board and manage the code.

iii) Circuit Setup

DHT22 sensor is connected to digital pin 7 for data reading, with its **VCC** and **GND** connected to the 5V and GND pins of the Arduino.

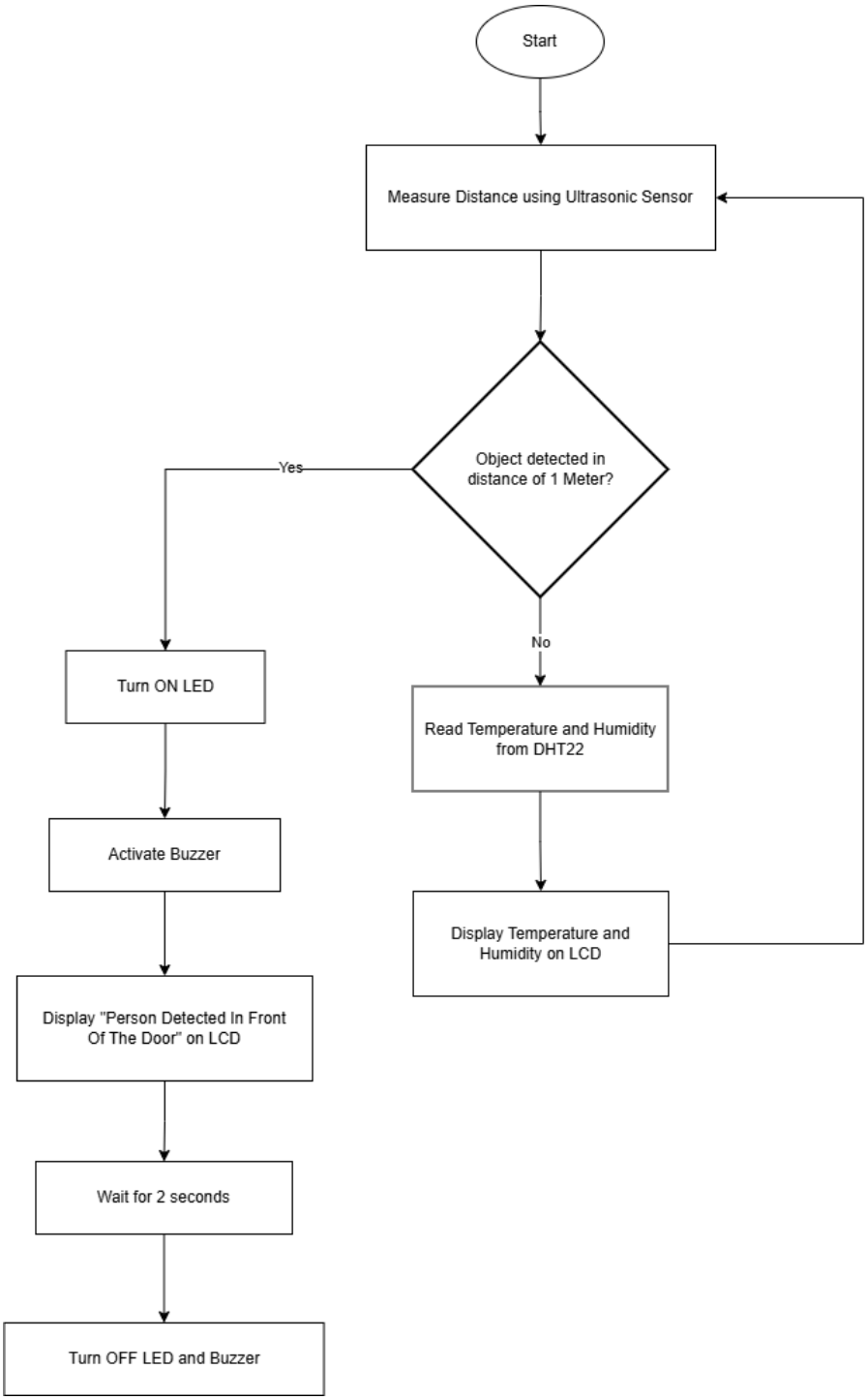
The **I2C LCD display** is connected to the **SDA** (A4) and **SCL** (A5) pins of the Arduino for data communication, with the **VCC** and **GND** connected to the 5V and GND of the Arduino



The **buzzer** is connected to digital pin 8, with a **resistor** in series to limit current, and the other leg is connected to GND.

The **TRIG** and **ECHO** pins of the **HC-SR04 ultrasonic sensor** are connected to digital pins 9 and 10 of the Arduino, respectively.

3.0 FLOWCHART



4.0 CONCLUSION & DISCUSSION

The "**Smart Door Alarming System**" demonstrates a practical and effective use of IoT technologies to enhance home security. The system's ability to detect objects near a door and trigger visual and audible alarms provides a strong first layer of protection for homeowners. The integration of real-time temperature and humidity monitoring adds an additional layer of functionality, making the system not just a security device but also a basic environmental monitor. However, this versatility comes with limitations, as the ultrasonic sensor detects any object within its range, including non-human entities like pets, furniture, or other obstacles. This could occasionally result in false alarms, highlighting a potential area for improvement in refining detection mechanisms.

The inclusion of temperature and humidity readings serves a dual purpose. First, it allows users to monitor environmental conditions in real time, which could be particularly useful for ensuring a comfortable and safe living environment. Second, these readings may provide indirect insights into security which for instance, unusual environmental changes could indicate door tampering or unauthorized entry attempts. While these features increase the system's utility, they also demand a certain level of user understanding and maintenance, such as regularly calibrating sensors for optimal accuracy.

To further enhance the system, additional sensors and modules could be integrated. A motion detection sensor, such as a Passive Infrared (PIR) sensor, could distinguish between humans and non-humans, reducing false alarms. A camera module could enable real-time visual monitoring, while Wi-Fi or GSM modules could facilitate remote notifications and controls. These upgrades would make the system more robust and user-friendly, allowing homeowners to respond promptly to potential security breaches. Future iterations could also explore machine learning models to analyze sensor data and improve the accuracy of detection and alerts.

Developing and implementing this project highlights the critical role of both software and hardware knowledge for computer science students. Working with microcontrollers, sensors, and coding environments such as the Arduino IDE bridges the gap between theoretical concepts and real-world applications. Such projects foster essential problem-solving skills and provide hands-on experience in integrating software logic with hardware components.

5.0 REFERENCES

Arduino. (n.d.). *Arduino - Home*. Retrieved January 9, 2025, from https://www.arduino.cc/

Codebender. (n.d.). *Codebender: Arduino IDE*. Codebender. Retrieved January 9, 2025, from https://codebender.cc/

Sofroniou, G. (2018, May 11). Wireless Doorbell - Arduino - XBee Shield - PIR motion sensor - Final version [Video]. YouTube. https://youtu.be/VFI7EMF10JM?si=F0d8ocFRiiaLWj95

Arduino. (2023, June 26). Arduino UNO R4: The New Dimension of Making [Video]. YouTube. https://youtu.be/XWrAjvwHOr8?si=mwT3XZc80faAxlJ3